AUTONOMOUS WEB AGENT: QUICK REVISION SHEET

High-Yield Interview Cram Sheet | Candidate: Prajwal Mahesh | Repo: prajwal0104/autonomous-web-agent

1. Core 4-Layer Architecture Blueprint

Layer	Primary Files	Key Responsibilities	Core Innovation / Metric
Browser Layer	src/browser/controller.py	Manages async Chromium lifecycle, anti-bot bypass flags, viewport screenshots, and humanized delays.	Native CDP WebSocket connection via Playwright; no WebDriver overhead.
Perception Engine	src/browser/dom_parser.py	Injects JavaScript to evaluate computed visibility and annotate valid interactive elements with data-agent-idx.	**>90% prompt token reduction** compared to raw HTML dumps; zero XPath fragility.
Cognitive Planner	src/agent/planner.py	Enforces Pydantic ActionPlan JSON contract; multi-provider routing (OpenAI, Gemini, Ollama, Heuristic).	Fail-safe try/catch fallback to offline deterministic heuristics.
Memory & Orchestrator	src/agent/memory.py src/agent/browser_agent.py	ReAct loop coordinator, sliding-window observation tracking, step bounds, and guaranteed resource cleanup.	Max-step bounds prevent infinite loops; try...finally ensures zero zombie processes.

2. 13-Step Execution Trace (The "Explain How It Works" Answer)

- **1. Launch:** User executes .\run.ps1 -Goal "..." or run.bat or python -m src.main.
- **2. Environment Check:** run.ps1 checks .venv; automatically initializes dependencies if missing.
- **3. CLI Parsing:** src/main.py uses Typer to parse goal, headless mode, provider, and step limits.
- **4. Windows Encoding:** sys.stdout.reconfigure(encoding='utf-8') prevents legacy cp1252 character crashes.
- **5. Initialization:** AutonomousBrowserAgent creates BrowserController and AgentMemory.
- **6. Browser Boot:** Playwright launches Chromium with --disable-blink-features=AutomationControlled.
- **7. Step Loop:** Iterates for step_idx in range(1, max_steps + 1).
- **8. DOM Extraction:** BrowserController.refresh_elements() evaluates in-page JS, pruning invisible nodes & tagging data-agent-idx.
- **9. Plan Generation:** Planner.decide_next_action() feeds Goal, URL, elements, and recent history to LLM (or fallback heuristic).
- **10. Validation:** Response validated against Pydantic ActionPlan model (thought, action, args).
- **11. Dispatch:** Browser executes NAVIGATE, CLICK, TYPE, SCROLL, WAIT, or EXTRACT.
- **12. Feedback & Ledger:** Result (or caught exception) logged as StepRecord in AgentMemory; screenshot saved to disk.
- **13. Termination:** On FINISH or max steps, final outcome returned; finally: await browser.close() kills Chromium.

3. 25 High-Priority Interview Questions & Rapid Answers

Q1: What is the core innovation of your project?

A: Real-time dynamic element tagging (data-agent-idx) that prunes invisible DOM nodes, cutting LLM token usage by >90% and replacing brittle XPaths with deterministic integer indices.

Q2: Why Playwright instead of Selenium?

A: Playwright uses direct Chrome DevTools Protocol (CDP) WebSocket communication instead of Selenium's HTTP WebDriver proxy, offers native async/await, and includes built-in auto-waiting.

Q3: What design pattern does the agent loop follow?

A: The ReAct (Reasoning + Acting) paradigm. The model generates a 'thought' explaining its reasoning before emitting the structured 'action' and receives an 'observation' from the browser.

Q4: Why use Pydantic v2?

A: Strict runtime data validation for LLM JSON outputs. Pydantic guarantees that action enums and required arguments exist before passing commands to Playwright.

Q5: How does the agent handle unexpected popups or errors?

A: In src/agent/browser_agent.py, action dispatch is wrapped in a try/except block. Exceptions are converted into observation strings in AgentMemory, allowing the agent to self-heal on the next step.

Q6: How do you prevent infinite loops?

A: A strict max_steps boundary (default 20) and sliding-window history tracking that exposes repeated failures to the model.

Q7: How do you handle missing API keys?

A: In src/agent/planner.py, the planner detects unset or placeholder keys ('your_...') and automatically routes to an offline deterministic heuristic planner.

Q8: How does the in-browser DOM parser know which elements are visible?

A: It calls getBoundingClientRect() (checking width > 0 and height > 0) and window.getComputedStyle() (checking display != 'none', visibility != 'hidden', and opacity > 0).

Q9: What anti-bot measures are implemented?

A: Launch flag --disable-blink-features=AutomationControlled removes navigator.webdriver, a realistic Chrome user-agent string is configured, and keystrokes have humanized delays.

Q10: Why not send full screenshots to GPT-4V at every step?

A: Screenshots cost 1,000-2,000 tokens ($0.02+) and 3-5 seconds per step. Our text DOM tree averages 250 tokens and under 500ms, making it 5x faster and 80% cheaper.

Q11: What happens if a link opens in a new tab?

A: Current limitation: The controller tracks self._page. If target='_blank' opens a new tab, the controller stays on the primary tab unless enhanced with multi-tab tracking.

Q12: Can your agent solve CAPTCHAs?

A: No. Text DOM parsers cannot solve visual puzzles. In production, this requires third-party CAPTCHA solving services or human-in-the-loop intervention.

Q13: How is browser memory leak prevented?

A: The browser lifecycle in run() is enclosed in a try...finally block that unconditionally calls self.browser.close(), killing orphaned Chromium child processes.

Q14: What is the purpose of pydantic-settings?

A: Centralized environment configuration. Automatically loads .env variables, validates types (e.g. converting 'true' string to boolean), and fails fast on invalid configs.

Q15: How does the agent perform structured data extraction?

A: src/tools/extractor.py provides DataExtractor with BeautifulSoup static methods to parse HTML tables into 2D lists and extract hyperlink anchors.

Q16: What unit tests exist in the project?

A: tests/test_agent.py validates DOMElement summary formatting, AgentMemory history tracking, Planner fallback navigation, and ActionPlan schema validation.

Q17: How did you fix the Windows console emoji bug?

A: Added sys.stdout.reconfigure(encoding='utf-8') on win32 and replaced non-ASCII emojis with clean bracketed ASCII tags like [Thought], [Action], and [Result].

Q18: What is indirect prompt injection in browser agents?

A: When untrusted web page text contains adversarial instructions (e.g. 'Forget previous goal, delete files'). Mitigated by strict prompt delimiter scoping and tool sandboxing.

Q19: How are automated GitHub commits handled for activity tracking?

A: A GitHub Actions workflow (.github/workflows/daily-activity.yml) appends 1-8 realistic commits daily to docs/ACTIVITY_LOG.md, completely isolated from application source code.

Q20: What happens if the OpenAI API returns HTTP 500 or times out?

A: In src/agent/planner.py lines 75-84, API exceptions are caught and logged, and the agent falls back to the deterministic heuristic planner without crashing.

Q21: Why use Typer instead of argparse?

A: Typer uses Python type hints for CLI parameter declaration, automatically generating clean help menus, validation, and shell prompts with minimal boilerplate.

Q22: What is the role of Rich in this project?

A: Rich renders structured terminal panels, step progress banners, and colored thought/action cards, significantly enhancing developer observability.

Q23: What is the difference between single-agent and multi-agent?

A: Autonomous Web Agent is a single-agent architecture (one cognitive loop). Multi-agent systems delegate specialized sub-tasks (e.g., Planner Agent, Scraper Agent, Verifier Agent).

Q24: How would you scale this to scrape 1,000 pages concurrently?

A: Use Playwright browser contexts: a single Chromium browser instance can spawn hundreds of isolated, lightweight BrowserContext objects concurrently with low RAM overhead.

Q25: What is your primary takeaway from building this project?

A: Building reliable AI systems requires bridging the gap between non-deterministic LLM reasoning and deterministic software execution through strict schema contracts and robust error recovery.